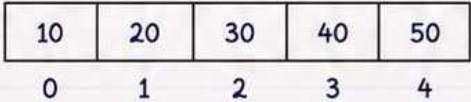
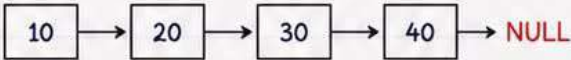
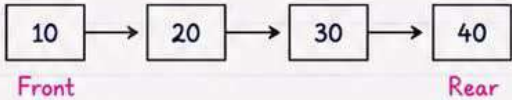
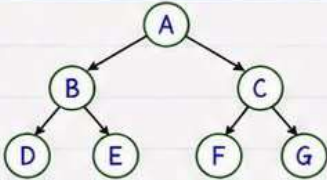
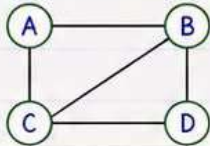
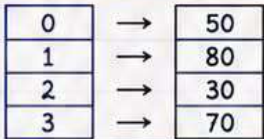
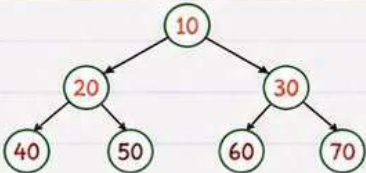


≧ Data Structures ≦

Definition

A data structure is a way of organizing and storing data so that it can be used efficiently. It helps in optimizing the performance of programs by allowing efficient access, modification, and processing of data.

All Data Structures Overview

<u>Data Structure</u>	<u>Diagram / Representation</u>	<u>Description</u>
1. Array		Stores elements of the same type in contiguous memory locations.
2. Linked List		Stores elements in nodes. Each node points to the next node.
3. Stack (LIFO)		Follows LIFO (Last In, First Out) principle. Insertion & deletion at the top.
4. Queue (FIFO)		Follows FIFO (First In, First Out) principle. Insertion at rear, deletion at front.
5. Tree		Hierarchical structure with a root node and zero or more subtrees.
6. Graph		Collection of nodes (vertices) connected by edges.
7. Hash Table (Hash Map)		Stores key-value pairs using a hash function for fast access.
8. Heap		A special tree-based structure (Min-Heap or Max-Heap).

Pro Tip:

Understanding the right data structure is the key to writing efficient and optimized code. ☆

≧ What is an Algorithm? ≦

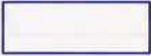
Definition

An algorithm is a finite set of well-defined instructions or steps to solve a problem or accomplish a task. It must produce output for given input and terminate after a finite number of steps.

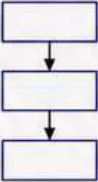
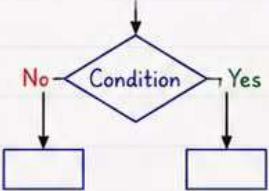
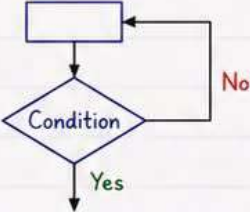
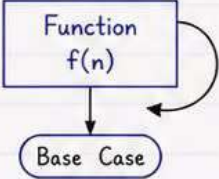
Characteristics of an Algorithm

1. Input : Takes zero or more inputs.
2. Output : Produces at least one output.
3. Definiteness : Each step is clear and unambiguous.
4. Finiteness : Terminates after a finite number of steps.
5. Effectiveness : Each step is basic and executable in finite time.

Basic Symbols in Algorithms / Pseudocode

<u>Symbol</u>	<u>Name</u>	<u>Use</u>
	Terminator	Indicates start or end of the algorithm.
	Input / Output	Represents input or output.
	Process	Represents an instruction or step.
	Decision	Represents a decision point with Yes/No (True/False) paths.
	Flow Line	Shows the direction of flow.
	Connector (On-page)	Connects flow lines on the same page.
	Connector (Off-page)	Connects flow lines to another page.

Types of Algorithms

1. Sequence Steps are executed one after another. 	2. Selection (Decision) Choice is made based on a condition. 	3. Iteration (Loop) Steps are repeated until a condition is met. 	4. Recursion A function calls itself to solve smaller instances of the same problem. 
---	--	---	--

Pro Tip:

A good algorithm is correct, efficient, simple to understand, and solves the problem with optimal time and space complexity.



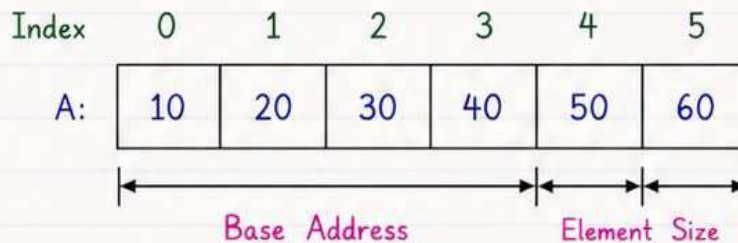
Arrays

Definition

An array is a collection of elements of the same type stored at contiguous memory locations. Each element can be accessed using an index.

Representation

Array: A



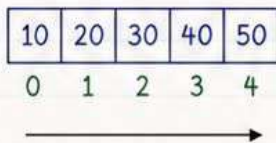
Characteristics

1. Fixed Size : Size is fixed once declared.
2. Homogeneous : All elements are of the same type.
3. Contiguous Memory : Elements are stored in adjacent memory locations.
4. Random Access : Any element can be accessed directly using its index.
5. Efficient Access : Access time is $O(1)$ for any element.

Basic Operations

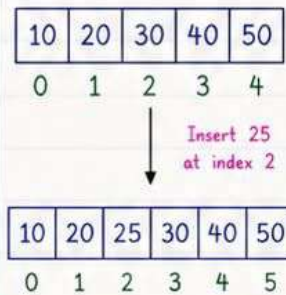
1. Traversal

Visit each element of the array.



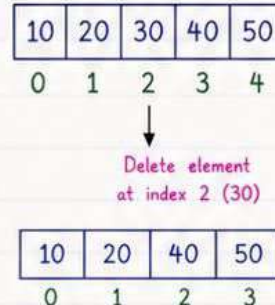
2. Insertion

Add an element at a specified position.



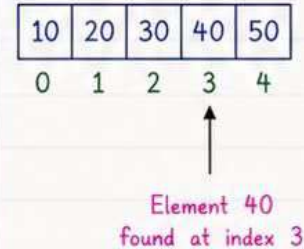
3. Deletion

Remove an element from a position.



4. Searching

Find the index of a given element.



Applications

- Storing lists of data such as marks, scores, names, etc.
- Implementing other data structures like stacks, queues, heaps, etc.
- Used in searching, sorting, matrix operations, lookup tables, etc.

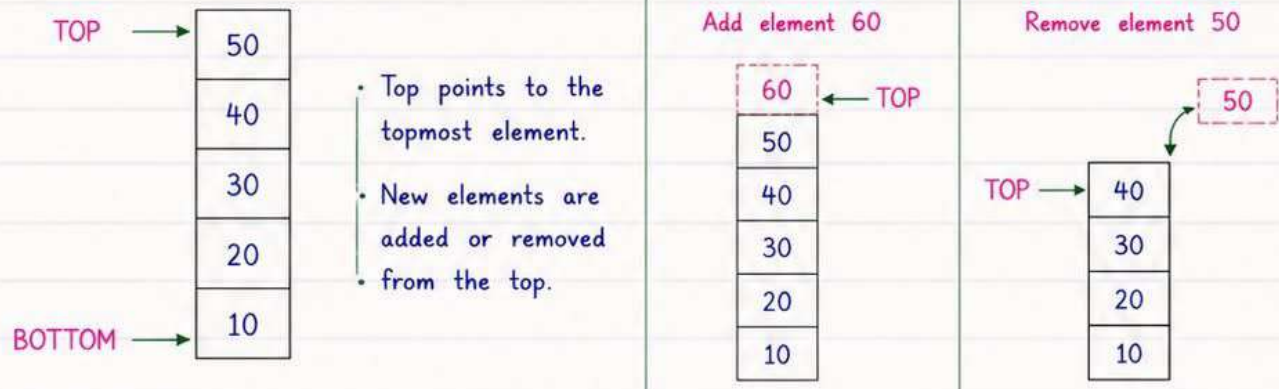
Pro Tip: Arrays provide fast access to elements by index, but inserting or deleting in the middle can be costly as elements need to be shifted. ★

≧ Stacks ≦

Definition

A stack is a linear data structure that follows the **LIFO** (Last In, First Out) principle. Elements are added and removed from the same end, called the top.

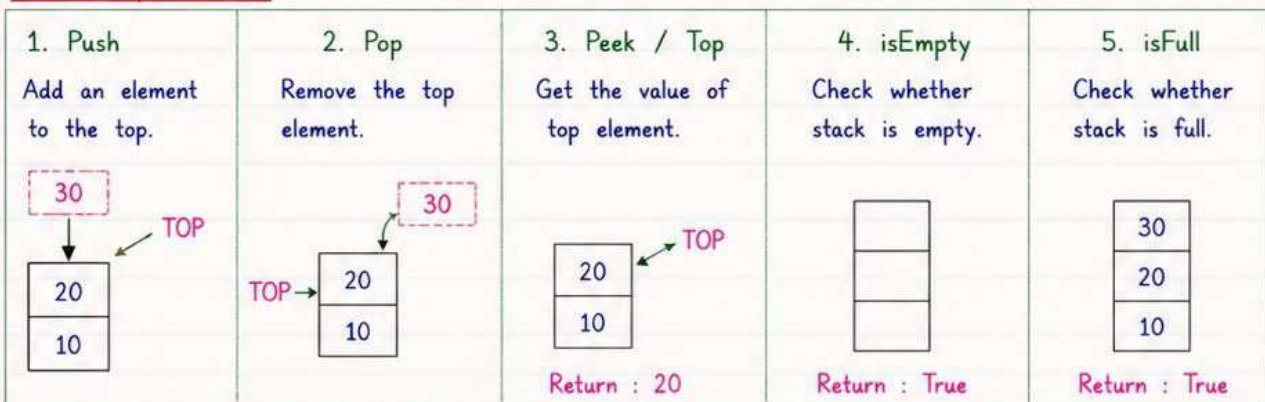
Representation



Characteristics

1. Follows LIFO (Last In, First Out) principle.
2. Insertion and deletion occur only at the top.
3. **Dynamic size** : Can grow or shrink during program execution.
4. **Efficient operations** : Push and pop are $O(1)$.
5. Used in function calls, expression evaluation, backtracking, etc.

Basic Operations



Applications

- Function call management (Call Stack).
- Expression evaluation and conversion (infix $\rightleftharpoons$ postfix/prefix).
- Parentheses matching and syntax parsing.
- Backtracking (e.g., maze, N-Queens problem).
- Undo operations in editors.
- Browser history navigation.

Pro Tip :

Always make sure to check for overflow before push and underflow before pop to avoid errors!

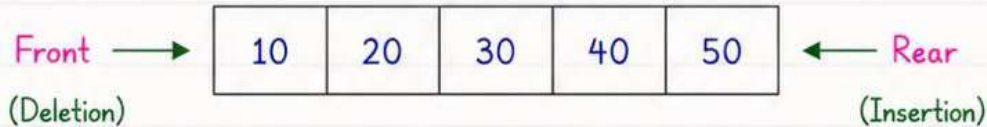


Queues

Definition

A queue is a linear data structure that follows the FIFO (First In, First Out) principle. Elements are added at the rear and removed from the front.

Representation



Front indicates the first element (to be removed).

Rear indicates the last element (where new element is added).

Characteristics

1. Follows FIFO (First In, First Out) principle.
2. Insertion occurs only at the rear.
3. Deletion occurs only at the front.
4. Dynamic size : Can grow or shrink during execution.
5. Efficient operations : Enqueue and Dequeue are $O(1)$.

Basic Operations

1. Enqueue	2. Dequeue	3. Front/Pek	4. isEmpty	5. isFull
Add an element at the rear.	Remove the element from the front.	Get the value of front element.	Check whether the queue is empty.	Check whether the queue is full (for fixed-size).
 Front ↓ Rear ↓ [10 20 30 40]	 Front ↓ Rear ↓ [10 20 30] ↓ [10]	 Front ↓ Rear ↓ [10 20 30] ↓ [10] Return : 10	 [] [] [] [] Return : True	 [10 20 30 40] Return : True

Applications

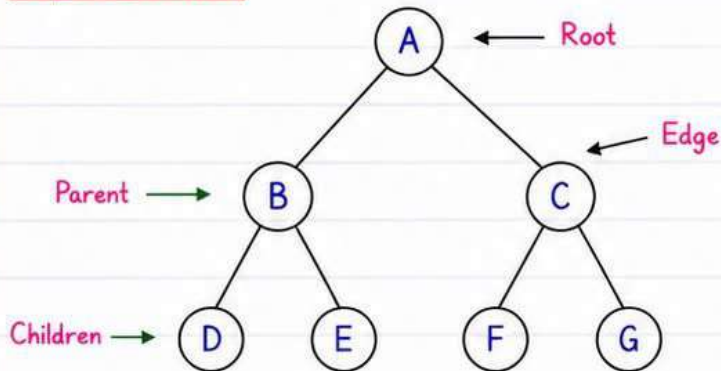
- CPU scheduling and task management.
- Handling requests in web servers.
- Printer queue (print spooling).
- Breadth-First Search (BFS) in graphs.
- Simulation of waiting lines (e.g., ticket booking, call centers).

Trees

Definition

A tree is a non-linear data structure consisting of nodes connected by edges. It has a hierarchical structure with a single root node and zero or more subtrees.

Representation



- **Root** : Topmost node in the tree.
- **Edge** : Connection between two nodes.
- **Parent** : A node that has one or more children.
- **Child** : A node that is connected to a parent node.
- **Leaf Node** : A node with no children.
- **Subtree** : A tree formed by a node and all its descendants.

Characteristics

1. A tree has exactly one root node.
2. Each node (except the root) has exactly one parent.
3. A node can have zero or more children.
4. There are no cycles in a tree.
5. A tree with n nodes has exactly $n - 1$ edges.

Basic Operations

1. Traversal	2. Insertion	3. Deletion	4. Searching	5. Finding Parent / Child
Visit each node in a specific order.	Add a new node to the tree.	Remove a node from the tree.	Find a node with a given value.	Identify parent or children of a node.
Types : • Preorder • Inorder • Postorder • Level Order	 Insert F	 Delete E	 Search C	 Parent of B is A Children of B : D, E

Applications

- Representing hierarchical data (e.g., file systems, organization charts).
- Expression trees in compilers.
- Database indexing (e.g., B-trees).
- Decision making (e.g., game trees, AI).

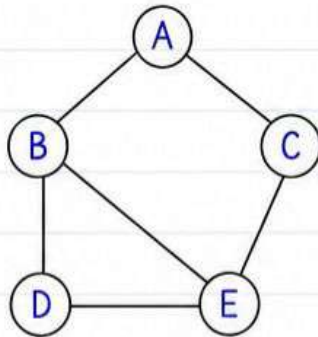
≡ Graphs ≡

Definition

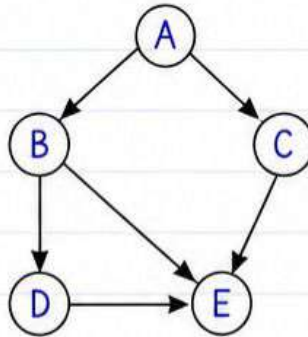
A graph is a non-linear data structure consisting of vertices (nodes) and edges. It can be directed or undirected.
Graphs can be used to represent networks and relationships.

Representation

Undirected Graph



Directed Graph

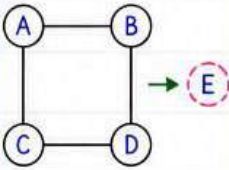
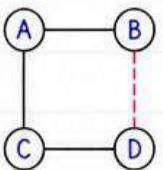
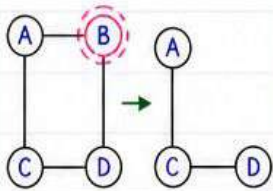
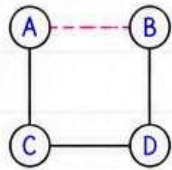
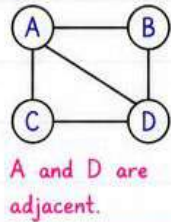


- **Vertex (Node)** : An individual point in the graph.
- **Edge** : A connection between two vertices.
- **Directed Graph** : Edges have a direction.
- **Undirected Graph** : Edges have no direction.
- **Weighted Graph** : Edges have associated weights or costs.
- **Connected Graph** : There is a path between every pair of vertices.

Characteristics **Save & Follow**

1. A graph consists of a set of vertices (V) and a set of edges (E).
2. Edges can be directed (one direction) or undirected (no direction).
3. A graph can be weighted or unweighted.
4. A graph can be connected or disconnected.
5. A simple graph has no self-loops and no parallel edges.
6. Used to model real-world networks and relationships.

Basic Operations

1. Add Vertex Add a new vertex to the graph.	2. Add Edge Add an edge between two vertices.	3. Remove Vertex Delete a vertex and its incident edges.	4. Remove Edge Delete an edge between two vertices.	5. Check Adjacency Determine if two vertices are adjacent.
				

Applications

- Social networks (friends, followers, connections).
- Road maps and navigation systems.
- Network design and communication systems.
- Recommendation systems.